

Capture the Flag: Designing and Deploying Realistic Web Exploitation CTF Challenges Using Modern Frameworks

Student Name: Pulindu Fonseka

Supervisor Name: David Oswald

Submitted as part of the degree of MEng Computer Science to the
Board of Examiners in the Department of Computer Sciences, Durham University

Research Question:

"How can modern web application vulnerabilities be authentically modelled and pedagogically structured within Capture-the-Flag environments to enhance learning outcomes in cybersecurity education?"

1 Project Plan

1.1 Project Description

CAPTURE-THE-FLAG (CTF) competitions have become a cornerstone of practical cybersecurity education. Studies have shown that CTF-style exercises greatly increase student engagement and skill development by simulating real scenarios in a hands-on, game-like environment. In a typical CTF, participants solve challenges across domains (including web, binary, cryptography, reverse engineering, forensics, etc.) to find hidden "flags" [1]. Educational initiatives like picoCTF [2] have provided key infrastructure to enable the scalability of CTFs for teaching thousands of students foundational hacking skills. However, integrating CTFs into coursework requires careful design to ensure pedagogical effectiveness and fairness. CTFs are recognized as effective for promoting cybersecurity awareness and hands-on skill refinement.

This project was proposed to improve the current Cyber Security sub-module in Networks and Systems (COMP2211) at Durham University [3]. A summary of the coursework is that it is a Jeopardy-style CTF challenge, similar to Hack the Box [4] or Over the Wire [5], where students receive nearly identical VMs containing a number of challenges, with the completion of each task awarding marks. The only difference to prevent cheating is randomised flags per student. While effective for hands-on learning, the existing system has vulnerabilities (for example, if a student can gain root access, they can bypass challenges easier, and the uniform structure makes it susceptible to AI-assisted and peer-to-peer cheating). This dissertation will focus on increasing the difficulty of the challenges, by adding new challenges, with more modern vulnerabilities to provide a greater challenge. It is important to note that despite the advancements made, there remain several issues including the identity of the challenges which future work on will provide a great benefit for the coursework. The coursework is comprised on a VM, and once logged in, there are 23 CTF jeopardy-style problems, with each correct flag awarding points. There is a range of problems including SQL injection, cryptography and Pwning. Our paper will focus on realistic Web Exploitation and Web Application Security, where players will find and explore vulnerabilities on web applications. The current coursework's web challenges are built in PHP, with some client-side JavaScript. We wanted to expand on this by utilising more common modern languages and frameworks including Java and Node.js. We will have 3 main types of challenges (basic, intermediate and advanced), with progressively increasing difficulty and complexity and realism.

Furthermore, there is opportunity to gain data from willing student participants in the current Networks and Systems module in January-February 2026. We can utilise this opportunity by providing the students with this optional project for those interested to gain feedback.

1.2 Project Deliverables

1.2.1 Basic Deliverables

The basic deliverables will focus on one main stack; we will implement 3 basic jeopardy-style CTFs with an easy-medium difficulty (in relation to the existing coursework) in the following topics:

- PHP legacy app: Classic PHP pages with MySQLi, with an SQL injection vulnerability.
- Node.js: Where there is an insecure express app, with a focus on routing, insecure middleware or file uploads. The learning outcome will be Node debugging and learning request/response injection. We may also use the Express framework.
- React: Simple modern, single page app with backend API storing user input. The learning outcome will be a DOM XSS discovery.

1.2.2 Intermediate Deliverables

The intermediate deliverables may have 2-3 topics with multiple steps required. We may also use 'bread crumbing', where an exploit may lead to another exploit. We will have the following challenges:

- Java Spring Boot: a modern insecure Spring Boot framework.
- Flask: SSTI and RCE challenge.
- Static File Server: path traversal.

1.2.3 Advanced Deliverables

The advanced deliverables will be a further step up, where it will combine multiple topics and require several steps for a player to obtain the flag. The aim is to complete 1-3 different advanced CTF levels. The levels will feature multi-stage chain attacks in a realistic enterprise scenario.

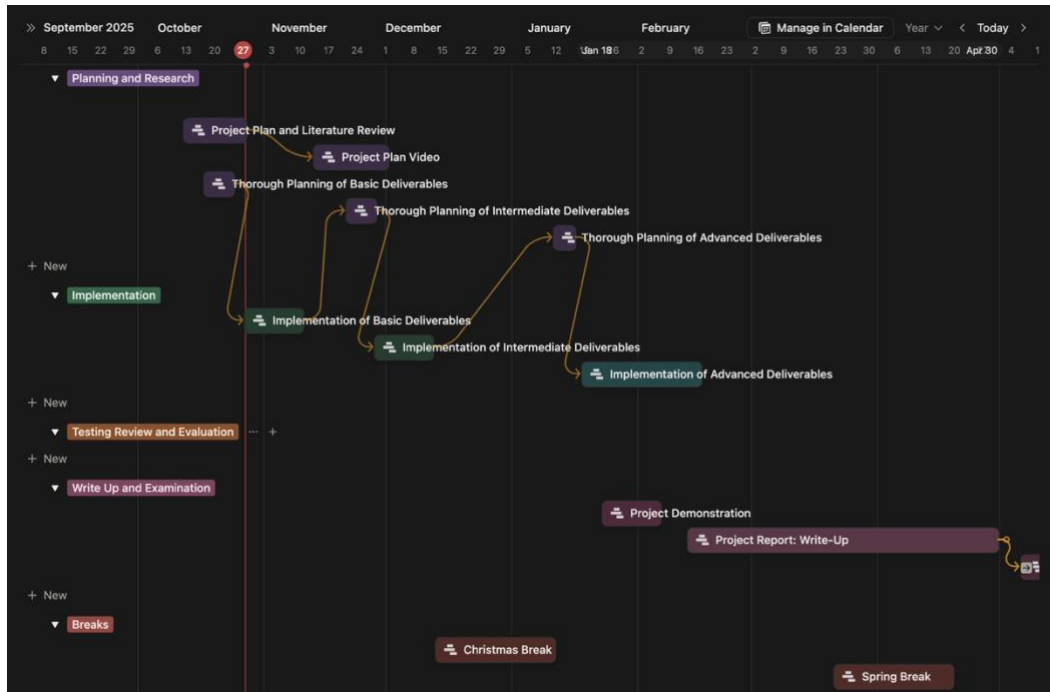


Figure 1: GANTT chart indicating timescales for objectives and deliverables.

1.3 Timeline / GANTT Chart

2 Literature Survey

2.1 Designing CTF Challenges in Cybersecurity Education

Various literature offers insights and best practices for crafting engaging challenges. A well-designed CTF set will typically cover multiple domains of security and gradually increase in difficulty to accommodate beginners while still challenging experts. Balaji *et al.* (2025) [6] describe an educational CTF platform they built which includes multi-level challenges in cryptography, reverse engineering, file forensics, and web exploitation. Each level ends with a flag that unlocks the next, creating a structured learning path from basic to advanced topics. This progressive difficulty ensures inclusivity, newcomers can solve easier puzzles and learn fundamentals, while seasoned players face increasingly complex tasks later. Such multi-tiered CTF design is noted to promote continuous learning and keep participants of all skill levels engaged [6]. This relates to our project because it is important to build up the levels, we will therefore begin our project by building few 'easy' levels that are accessible to most, and progress by building more challenging levels as the project progresses.

The report "Effectiveness of CTF Education" Meinsma *et al.* (2022) [7] provides a measurement framework for Jeopardy-style CTF learning outcomes, applying it to two competitions hosted on CTFd [8]. They provide a rare framework for measuring learning outcomes in CTFs, linking challenges to explicit statements of *knowledge*, *skill*, and *attitude* and offering a practical checklist for educators. The study sits within European cyber-education initiatives and treats CTFs as structured assessments rather than informal games, using real competition data to examine whether participants "actually learn". However, results show no clear evidence of measurable learning growth, performance and enthusiasm did not always align, and logging data could not reveal *how* or *why* learning occurred. This challenges assumptions that CTF participation automatically builds competence. For this project, the paper reinforces the need to embed measurable learning objectives within each web-exploit challenge (e.g., SQLi, XSS) and to capture process-based evidence of skill development, not just flag completion. It

underlines the importance of balanced difficulty and clear educational scaffolding when designing advanced, web-focused CTF tasks.

“The Many Maxims of Maximally Effective CTFs” P. Fuzyll (2015) [9] provides a practitioner-driven framework for effective challenge design. It defines enduring principles (realism, fairness, reproducibility, and the idea that learning should be the path to the flag) which remain core to CTF development. Unlike academic studies, it captures the community’s applied experience and complements research on CTF pedagogy and infrastructure. However, this informal, entertainment-focused tone lacks empirical backing and sometimes conflicts with educational approaches that priorities scaffolding or feedback. For this project, it underpins the design ethos: build realistic, contained web-exploit challenges (e.g., SQLi, XSS) that reward understanding rather than guesswork, ensuring tasks are both educational and technically authentic. Although not completely backed with evidence, it should be a useful guide in designing my CTF challenges.

2.2 CTF Platforms and Hosting

Although Containerization isn’t the focus of our dissertation, we have decided to include this section for in the event that an advanced deliverable for the CTF creation requires it, we have a high-level overview on it.

When creating a CTF for an assignment or event, one must consider how to host and deliver the challenges. Traditionally, many CTFs were distributed as a single virtual machine (VM) image or hosted on isolated VMs. However, there is a clear trend toward using web-based CTF platforms and containerization for scalability. Various open-source CTF frameworks exist (CTFd, picoCTF’s platform, iCTF [8] framework, etc.), which provide the infrastructure for user registration, challenge listing, flag submission, and scoreboards [10] [11]. Karagiannis *et al.* (2020) [10] performed a comparative evaluation of four popular open-source CTF platforms, examining features like usability, extensibility, challenge management, and learning support. Participants in their study valued features such as hint integration, good challenge categorization, and an intuitive interface, which all enhance the learning process. The takeaway is that the choice of platform can impact the learner’s experience, a robust platform with the right features allows organizers to focus on creating quality challenges rather than worrying about basic functionality. Furthermore, a robust, straightforward platforms allow the player to focus more on the challenges than spending time on figuring out the structure.

Modern CTF deployments often use Docker containers to host each challenge service in the cloud, rather than requiring players to run a full VM locally. This approach is lighter-weight and more scalable. Raj *et al.* (2016) [12] introduced a CTF infrastructure that uses Docker containers instead of virtual machines, finding it “*significantly reduces the resource requirements*” for organizers and automates many setup tasks. Because containers share the host OS kernel, running many instances is far more efficient than spawning multiple full VMs. The Docker-based approach let organizers handle more simultaneous users and challenges with significantly lower hardware requirements, a single powerful computer could host the CTF for many participants. Another benefit is easier distribution and deployment: participants simply connect to a challenge’s container via a provided URL/IP, without needing to download large images or configure VMs. HackTheBox, for example, spawns a fresh Docker instance per challenge instance and gives players an IP and port to interact with, simplifying the experience [13]. Researchers note that containerization also simplifies reset and isolation, this also ensures that one player’s actions doesn’t affect another.— each team or player can get a clean instance of a vulnerable service, which is destroyed after use, ensuring one group’s actions don’t affect another [12]. This is particularly important for our project, since we want to ensure fairness amongst students.

For an academic project, exploring container-based deployment (using tools like Docker Compose, Kubernetes, or CTFd’s Docker integrations) could be highly beneficial. In summary, the literature and current practice suggest moving away from monolithic VMs towards flexible web platforms and containers for hosting CTF challenges, a shift that improves scalability, resource usage, and user experience.

Sasiwanit *et al.* (2025) [14] present the *Gambling CTF* platform, a container-based cybersecurity training system integrating CTFd [8] for challenge management. They used Docker for challenge isolation and VPN access control. The authors combine architectural design with empirical evaluation, using Apache JMeter stress, endurance, and spike tests (100–2,500 users) to quantify scalability across four web-focused challenges: Hidden Information, Brute Force, SQL Injection, and

Cross-Site Scripting (XSS). This pairing of reproducible architecture and performance data provides a solid reference for scalable, hands-on web-exploit training systems. Positioned within CTF-based pedagogy, the work localises global vulnerabilities to Thai government and educational domains compromised by gambling defacements. It affirms CTFd + Docker as an effective open-source foundation for structured cybersecurity learning environments. Despite its realism claims, the platform relies on simplified attack surfaces (no multi-factor authentication, rate-limiting, or input sanitisation) trading authenticity for accessibility. Furthermore, the load-testing results are confined to a single virtual-machine setup, which restricts external validity. The challenge set remains limited to classical vulnerabilities, omitting modern exploit classes such as server-side template injection, deserialization, or prototype pollution. For this dissertation, the study validates using CTFd + Docker as a baseline infrastructure and incorporating formal load-testing in evaluation. However, it also highlights opportunities to extend realism by embedding layered defences, multi-service interactions, and procedurally generated or adaptive web-exploit scenarios. Addressing these gaps positions the present research to advance the design of authentic, scalable web-exploit CTF challenges.

2.3 Challenge Design Inspiration and Web Vulnerabilities

PicoCTF [11] offered various insights. They stated that using classic vulnerabilities is acceptable (even expected) at the beginner level since the goal is to teach fundamental skills. However, a high-quality CTF challenge should not be a rehash of textbook examples and instead it should contain an element of discovery or a subtle trick that makes the solver think "Ah, I didn't realize you could do that!" [11]. For web challenges, novelty can be introduced by exploiting uncommon quirks or combining multiple steps. One design strategy is to center a challenge on a behavior or edge case in modern web tech. An example they gave is a challenge might leverage a quirk in HTTP headers (e.g. requiring a special User-Agent value to trigger hidden functionality) or abuse a serialization format bug in a web framework. Another might involve a path traversal via image file URLs (e.g. GET /images/?file=.../etc./passwd) or exploiting an upload feature to achieve remote code execution. Such scenarios teach students how violating seemingly minor security guidelines can lead to major flaws [11]. Overall, literature emphasizes that each challenge should have clear learning value – upon solving, the student has gained a concrete skill or insight, not just a flag [11]. For instance, after completing a SQL injection challenge, a student should understand techniques like UNION SELECT or Boolean blind extraction, rather than having blindly brute-forced a token. To ensure this, challenge prompts and post-solution explanations can highlight the intended lesson.

A useful source would be the OWASP Top Ten [15]. OWASP is an international non-profit organization that is dedicated to web security. They frequently update the Top Ten with outlines of major concerns for web application security and their risks. It will be useful to include various exploits from the top ten in our CTFs.

"Identifying and Mitigating Web Application Vulnerabilities: A Comparative Study of Countermeasures and Tools" by Sadat et al. (2024) [16] provides us with a survey of common web-application vulnerabilities, using open-source web vulnerability tools, alongside discussing countermeasure strategies. The paper confirms that classic OWASP's [17] Top 10 web flaws (especially SQL Injection, Cross-Site Scripting, and Broken Authentication) remain the most relevant for defensive countermeasures. Each map directly to compact, teachable CTF challenge designs and clear mitigation lessons we can assess. This information can be used to justify our focus on the chosen web-exploit challenges. The countermeasure mapping helps us understand the intended educational outcomes of each challenge ("understanding input sanitization" for XSS). The tool survey gives ideas for validating our challenge environments to test for vulnerabilities or for building challenge hints/clues around them. This can also be used for validation later to mitigate unintended exploits. Nawrocki et al. discussed similar topics in *"Vulnerabilities of Web Applications: Good Practices and New Trends"* [18]. The paper provides a practical OWASP-mapped survey, an OWASP Analyzer test app, experimental checks on specific countermeasures and a compact checklist for developers, which will be directly reusable as validation rubrics. This paper can also be used to justify the core exploit selection.

Meng et al.'s *"Secure Coding Practices in Java: Challenges and Vulnerabilities"* (2018) [19] set out to analyze secure-coding guidance in practice for Java security APIs. They analyzed Stack Overflow Q&A data to determine what vulnerabilities were common and listed them, which include insecure cryptography use (such as hashing rather than encryption or insecure key lengths). This paper can be used to inspire a medium-hard challenge with insecure API implementations.

Li et al. in their "*Mining Node.js Vulnerabilities via Object Dependence Graph and Query*" [20] will also be a useful reference. They introduce a context-sensitive static analysis tool for Node.js that constructs an Object Dependency Graph to represent JS objects, which can detect Node.js specific vulnerabilities. It can be used as a validator to detect unintended prototype pollution or issues before publishing. They also include a GitHub repository with the tool which we can use [21].

"*Towards Automatic Detection and Exploitation of Java Web Application Vulnerabilities via Concolic Execution guided by Cross-thread Object Manipulation*" by Huang et al. [22] focuses on automating exploit generation for Java web application's cross-thread dataflows. The reason for their choice in project was due to prior techniques failing to model Java's shared-object, multi-thread semantics. JAEX attempts to fill that gap. Although not directly relevant for our topic, we can take inspiration from this paper by (in our intermediate – advanced deliverables) including cross-thread, multi-request vulnerability challenges as they teach realistic exploitation reasoning and are well justified by JAEX's findings.

PortSwigger [23] has various guidance on their Web Security course. Notably, we took interest in their guidance on Insecure Deserialization [24]. Although only a high-level overview, it provides a good general foundation of Insecure Deserialization Attacks, which can be used in a level of our CTFs. Similarly, it has more advanced topics in their course including GraphQL API, Web cache poisoning, HTTP request struggling and more [23].

2.4 Existing CTF's for Inspiration

There are various CTFs online to gain inspiration from. From these CTFs I can find challenging levels and attempt to either recreate or improve them. Such sources include Over the Wire, Try Hack Me, Hack the Box [25] [26] [27]. It would be useful to play more of the challenges that these resources offer.

2.5 Performance Evaluation Metrics

Metric	Measurement Method	Rationale
Learning Outcome Alignment	Mapping each challenge to specific, measurable learning objectives (e.g. " <i>Understand and exploit a common SQL Injection vulnerability</i> "). For more advanced challenges, we may have several objectives.	Directly addresses the need to embed measurable learning objectives and combat the finding that participation doesn't automatically build competence.
Challenge Difficulty Progression	Analysis of Completion Rates and Time-to-Solve across the Basic, Intermediate, and Advanced tiers during student testing (Jan-Feb 2026).	Verifies the design principle of progressive difficulty, ensuring Basic challenges are accessible and Advanced challenges require more time/steps.
Student Feedback on Realism and Engagement	Anonymous Questionnaires/Surveys administered to willing players. Questions will focus on challenge authenticity (realism), clarity of instructions, and overall engagement/interest.	Assesses adherence to the practitioner-driven principle of realism and fairness in design.
Intended vs. Unintended Exploits	Post-mortem analysis of student solution paths and collected flags.	When testing, check for unintended exploits for each CTF. Ensures players followed the intended learning path to the flag.
Functional Correctness	Unit and Integration Testing for each challenge to ensure the vulnerable application functions correctly, only accepts the correct flag format, and the flag retrieval logic is sound.	Ensures the core task is achievable and the flag validation mechanism is robust.

3 Critical Comparison with LLM

To create the AI generated Literature Survey, I used Google Gemini 2.5 Flash Research Mode [28]. The prompt used can be found in *Appendix A*.

The AI-generated literature survey was structurally comprehensive, addressing each section detailed in my project plan (since the project plan was used in the prompt). However, it was more of an elaboration of the project plan rather than an independent literature review. This led to a disproportionate focus on anti-cheating and fairness, a priority in the prompt's overview, at the expense of detailed web implementation research. However, this remained my literature review's main weakness too, although not as much. While the AI provided some suggestions for randomization and offered more detail on deliverables (including specific, modern vulnerabilities like Spring4Shell), it critically lacked academic rigor, failing to provide valid evidence or papers to back these specific technical claims. The most significant finding regarding accuracy was the minimal overlap in sources: only one key paper, the Meinsma et al. study on CTF educational effectiveness [7], appeared in both bibliographies, confirming the AI's tendency toward hallucination and demonstrating that it performed limited external research beyond the keywords supplied in the prompt. The LLM showed that it can't "think for itself well", however, it is sufficiently able to add more detail to existing points.

REFERENCES

- [1] J. Z. J. Z. Zewen Huang, "mdpi," 25 June 2025. [Online]. Available: <https://www.mdpi.com/2076-3417/15/13/7159#:~:text=In%20the%20field%20of%20cybersecurity%2C,edge%20sharing%20in%20cybersecurity.> [Accessed 17 October 2025].
- [2] Carnegie Mellon University, "picoCTF," [Online]. Available: <https://picoctf.org.> [Accessed 21 October 2025].
- [3] Durham University, "Programme and Module Handbook, Module COMP2211: NETWORKS AND SYSTEMS," 2025. [Online]. Available: <https://apps.dur.ac.uk/faculty.handbook/2020/UG/module/COMP2211.>
- [4] Hack the Box, [Online]. Available: <https://www.hackthebox.com.>
- [5] Over the Wire, [Online]. Available: [https://overthewire.org/wargames/.](https://overthewire.org/wargames/)
- [6] R. ., V. ., A. ., S. P. Balaji, "DESIGN AND DEVELOP CTF CHALLENGE USING CRYPTOGRAPHY AND STEGANOGRAPHY," 2025. [Online]. Available: https://www.irjmets.com/uploadedfiles/paper/issue_1_january_2025/66523/final/fin_irjmets1744049659.pdf#:~:text=platform%20ai%20med%20at%20enhanci%20cybersecurity,project%20offers%20diverse%20interactions%20to.
- [7] S. B. M. G. M. B. D. Meinsma, "Effectiveness of CTF education," 2022. [Online]. Available: https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://www.ncsc.nl/binaries/ncsc/documenten/rapporten/2022/september/15/effectiviteit-van-ctf-educatie/TLP_CLEAR_NCSC_Report_Effectiveness_of_CTF_education.pdf&ved=2ahUKewi49qympr-QAxVaU0E.
- [8] CTFd. [Online]. Available: <https://ctfd.io.>
- [9] P. Fuzyll, "The Many Maxims of Maximally Effective CTFs," WayBackMachine, 12 17 2015. [Online]. Available: <https://web.archive.org/web/20201212081922/https://captf.com/maxims.html#:~:text=1,survival%20of%20a%20vulnerable%20server.>
- [10] E. M.-B. E. M. S. Karagiannis, "An Analysis and Evaluation of Open Source Capture the Flag Platforms as Cybersecurity e-Learning Tools," [Online]. Available: https://www.researchgate.net/publication/344372369_An_Analysis_and_Evaluation_of_Open_Source_Capture_the_Flag_Platforms_as_Cybersecurity_e-Learning_Tools.
- [11] picoCTF, "picoCTF," [Online]. Available: <https://picoctf.org/posts/2025-04-12-how-you-can-write-ctf-challenges.html#:~:text=role%20in%20shaping%20it,quality%20of%20a%20CTF%20challenge.>
- [12] B. A. S. P. a. K. A. A.S Raj. [Online]. Available: <https://www.usenix.org/system/files/conference/ase16/ase16-paper-raj.pdf#:~:text=widespread%20adoption,handles%20certain%20system%20administration%20tasks.>
- [13] Hack the Box, "CTF User's Guide," [Online]. Available: <https://help.hackthebox.com/en/articles/5200851-ctf-user-s-guide.>
- [14] P. T. P. L. C. Sasiwanit, "Gambling CTF Challenges for Developing Hands-on Cybersecurity Graduates," Sept 2025. [Online]. Available: https://www.researchgate.net/publication/395454191_Gambling_CTF_Challenges_for_Developing_Hands-on_Cybersecurity_Graduates?enrichId=rgreq-2451fb880a579752899c0ac8dd043214-XXX&enrichSource=Y292ZXJYQWdIOzM5NTQ1NDE5MTtBUzoxMTQzMtI4MTYzNDQ3NjAxN0AxNzU3NzlyODA.
- [15] OWASP, "OWASP Top Ten," [Online]. Available: [https://owasp.org/www-project-top-ten/.](https://owasp.org/www-project-top-ten/)
- [16] M. N. K. S. S.E. Sadat, December 2024. [Online]. Available: https://www.researchgate.net/publication/387979958_Identifying_and_Mitigating_Web_Application_Vulnerabilities_A_Comparative_Study_of_Countermeasures_and_Tools#full-text.
- [17] OWASP, "OWASP Juice Shop," [Online]. Available: [https://owasp.org/www-project-juice-shop/.](https://owasp.org/www-project-juice-shop/) [Accessed 25 October 2025].
- [18] J. K. M.T. Nawrocki, "Vulnerabilities of Web Applications: Good Practices and New Trends," December 2024. [Online]. Available: https://www.researchgate.net/publication/389655054_Vulnerabilities_of_Web_Applications_Good_Practices_and_New_Trends.
- [19] S. N. D. Y. W. Z. G. A. N. Meng, "Secure CodingPractices inJava: Challenges and Vulnerabilities," 2018. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3180155.3180201.>
- [20] M. K. J. H. Y. C. S. Li, "Mining Node.js Vulnerabilities via Object Dependence Graph and Query," [Online]. Available: https://www.usenix.org/system/files/sec22summer_li-song.pdf#:~:text=that%20ODGEN%20is%20able%20to,CVE%20identifiers%20for%20these%20vulnerabilities.
- [21] S. Li, "github.com/Song-Li/ODGen," [Online]. Available: <https://github.com/Song-Li/ODGen.>
- [22] L. Z. Y. L. P. D. Y. C. Y. Z. M. Y. X. Huang, "Towards Automatic Detection and Exploitation of Java Web Application Vulnerabilities via Concolic Execution guided by Cross-thread Object Manipulation," [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity25-huang-xinyou.pdf.>
- [23] PortSwigger, "Web Security Academy - All Materials," [Online]. Available: <https://portswigger.net/web-security/all-materials.>
- [24] PortSwigger, "Deserialization," [Online]. Available: <https://portswigger.net/web-security/deserialization.>
- [25] Over the Wire, "Over the Wire Wargames," [Online]. Available: [https://overthewire.org/wargames/.](https://overthewire.org/wargames/)
- [26] Try Hack Me, "Try Hack Me," [Online]. Available: <https://tryhackme.com.>
- [27] Hack the Box, "Hack the Box," [Online]. Available: <https://www.hackthebox.com.>
- [28] Google, "Gemini," [Online]. Available: <https://gemini.google.com/app.>
- [29] NICCS, "NICE Workforce Framework for Cybersecurity (NICE Framework)," 28 August 2025. [Online]. Available: <https://niccs.cisa.gov/tools/nice-framework.>
- [30] M. Csikszentmihalyi, "Flow: The Psychology of Optimal Experience," January 1990. [Online]. Available: https://www.researchgate.net/publication/224927532_Flow_The_Psychology_of_Optimal_Experience.
- [31] G. S. V. A. A. G. R. Chadha, "A Study on Exploit Development," [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10079387&tag=1.>

Appendix

(Appendix A)

The prompt used the project plan in Section 1, as well as the following text:

"The above is my project plan for my integrated master's dissertation. Using this, create an in-depth literature survey. Your literature survey should give an overview of published work relevant to your project, highlighting the key findings of papers discussed and explaining how these relate to your proposed project."

(Appendix B)

Below is the full contents of the LLM generated prompt provided by Gemini:

A Literature Survey on Advanced Pedagogical Design and Integrity in Capture-The-Flag Coursework

I. The Pedagogical Imperative of Capture-The-Flag Competitions

The integration of Capture-The-Flag (CTF) exercises into academic curricula represents a critical pivot from passive, theoretical instruction toward active, experiential learning, which is necessary to cultivate job-ready cybersecurity professionals. CTFs are recognized internationally as a cornerstone of practical cybersecurity education, simulating realistic intrusion scenarios under time pressure.¹

1.1. Theoretical Foundations: Experiential and Gamified Learning

Cybersecurity, as a discipline, demands sophisticated problem-solving capabilities, which are fundamentally best developed through hands-on interaction with the environment.² This educational philosophy aligns with the principles championed by John Dewey, who argued that practical problem-solving and theoretical teaching must proceed synergistically.² CTF sessions provide the interactive, laboratory-style environment essential for engineering and science fields, enabling students to interact directly with vulnerabilities and defenses.²

The competitive and gamified structure inherent in CTFs—featuring scoring, leaderboards, and hidden objectives—significantly enhances student engagement, motivation, and knowledge attainment compared to traditional, lecture-based methods.¹ This gamification has been shown to improve critical thinking skills and motivate continued, self-directed learning.⁴ Crucially, empirical evidence confirms that participation in CTF activities substantially improves students' self-confidence in executing, recognizing, and defending against cyberattacks.¹ This separation of skill-based learning from purely knowledge-based instruction is paramount for developing competence in a field defined by rapid technological change.¹

A compelling finding from advanced security training suggests that CTF participation moves beyond simple skill acquisition and acts as a catalyst for radical conceptual refinement. When students are required to apply theoretical knowledge immediately in a functional, high-stakes context—such as debugging an exploit chain—they are forced to confront and rectify foundational misunderstandings of complex underlying Computer Science concepts. For instance, successfully manipulating memory protection mechanisms or race conditions necessitates a true understanding of process scheduling and memory isolation, concepts often poorly retained from earlier Computer Organization or Systems courses.⁶ The immediate, practical feedback loop provided by CTF exercises transforms abstract theoretical principles into necessary, functional knowledge, thereby cementing a deeper and more robust understanding of system architecture.

1.2. Taxonomy of CTF Models and Course Integration

The existing coursework utilizes the Jeopardy-style CTF format, which involves solving discrete challenges across various categories (Web, Cryptography, Pwning) to find hidden "flags".⁷ This style is highly effective for large-scale educational deployment, as demonstrated by initiatives like picoCTF⁸, and is ideal for foundational training covering a broad spectrum of cybersecurity domains.⁷ Conversely, the Attack/Defense CTF model, while providing a more realistic, dynamic simulation where teams simultaneously exploit and patch servers, is generally reserved for advanced collegiate competitions or professional training due to its inherent complexity and high resource requirements.⁹

The successful integration of CTFs into the university curriculum requires careful platform selection. Platforms must balance customization needs with scalability and support. Options range from dedicated educational platforms like picoCTF, which offers a progressive learning path and educational resources suitable for beginners⁸, to highly customizable, open-source frameworks like CTFd.¹⁰ Open-source solutions afford high

customization capabilities—essential for tailoring challenges to the specific learning objectives of a module—but rely on community support and variable hosting scalability. Hybrid approaches, combining customizable open-source infrastructure with professional deployment support, offer flexible scalability while maintaining medium customization capabilities.¹² The selection of platform architecture must be critically assessed based on the project's stringent requirements for challenge personalization and randomization, as detailed in the subsequent section on academic integrity.

Furthermore, pedagogical design must strictly manage the student's mental state during the challenge. The literature suggests that the instructional designer must maintain an appropriate challenge-to-skill ratio to keep students in a state of cognitive "flow".¹³ Challenges perceived as too difficult relative to the participant's skill level can induce anxiety and frustration, whereas challenges perceived as too easy lead to boredom, undermining the educational objective.¹³ The project's proposed three-tiered difficulty structure (Basic, Intermediate, Advanced) is thus not merely an organizational tool, but a crucial pedagogical design principle intended to provide adequate scaffolding and maintain the optimal flow state for continuous skill development across the entire cohort.

II. Designing for Academic Integrity and Scalable Fairness

The current Networks and Systems module's CTF coursework, which utilizes "nearly identical VMs containing a number of challenges" and relies solely on randomized flags for cheating prevention, harbors significant vulnerabilities [Project Plan]. These vulnerabilities expose the assessment to two primary risks: easy sharing of exploit methodology among peers and susceptibility to sophisticated, AI-assisted solution generation.¹⁴ Therefore, mitigating these integrity risks is a fundamental requirement of the proposed coursework redesign.

2.1. The Vulnerability of Uniformity and External Assistance

The homogeneity of the existing challenges allows a student who solves the problem once—or obtains a solution from a peer or public write-up—to easily replicate the solution across different instances, bypassing the core learning objectives [Project Plan]. This issue is amplified by the maturation of Large Language Models (LLMs) and AI tools like ChatGPT. Uniform problem structures or questions that are analogous to publicly documented vulnerabilities can be solved directly by these tools, necessitating a critical shift in assessment design.¹⁴

To successfully counter AI exploitation, assessments must transition from tasks focusing on recall or simple pattern recognition to those requiring **higher-order thinking**.¹⁵ This involves designing unique assignments and challenges that demand the application of complex principles, the drawing of non-obvious conclusions, the diagnosis of novel problems, and the proposal of bespoke solutions—actions that inherently require individual critical thought and cannot be reliably automated by current AI paradigms.¹⁴

2.2. Strategies for Personalization and Environment Randomization

The most robust academic strategy for ensuring assessment integrity in hands-on environments is the deployment of personalized, dynamically generated lab environments.¹⁶ Systems such as SecGen have established a methodology for creating unique machines and network configurations tailored to individual students.¹⁶

This personalization extends far beyond simple flag randomization. A robust system uses a unique identifier (such as a hashed email address or student ID) as a seed to dynamically configure the environment.¹⁶ The randomization includes multi-dimensional variation across the system stack:

- 1. **Network and Service Configuration:** Network services can be deployed on different, unique ports for each student, and distinct user credentials may be generated.¹⁶
- 2. **Vulnerability Profile:** The generator can randomly select elements of the target machines, such as specific operating systems or the configuration strength of components, subtly altering the exploit path.¹⁶
- 3. **Personalized Answer Validation:** The generated environment must store answers specific to that environment's configuration, ensuring that only a flag derived from the student's unique seed will be validated upon submission.¹⁶

Given that the project requires the creation of modern, custom challenges (Java Spring Boot, Node.js, etc.) [Project Plan], the deep customization required to implement this SecGen-style, multi-dimensional randomization framework must constrain the architectural choice of the CTF platform. Commercial or pre-integrated solutions often impose limitations on the deep structural variability needed for effective integrity measures.¹² Therefore, an open-source or hybrid platform approach—such as a heavily customized CTFd deployment integrated with a custom challenge generator—is likely mandatory to support the rigorous integrity framework outlined by the literature.¹¹

Finally, academic integrity is bolstered not only through technical countermeasures but also through pedagogical restructuring. The motivation and opportunity for cheating are substantially reduced when the grading structure avoids single high-stakes assessments.¹⁵ Instead, course points should be distributed across frequent, low-stakes assessments and a variety of methods, ensuring the CTF challenge itself is not the sole determinant of success.¹⁷ This strategy encourages students to view the challenges as practice and learning opportunities, rather than insurmountable obstacles requiring illicit assistance.

The following table summarizes the proposed framework for mitigating the academic risks inherent in uniform CTF coursework design.

Table 2: Mitigating Academic Integrity Risks in CTF Coursework

Identified Risk (Current System Vulnerability)	Research-Backed Mitigation Strategy	Literature Support
Identical Challenges/Shared Solutions (Uniform Structure)	Personalized Lab Environments (Unique seeds for system configs, ports, credentials)	SecGen Methodology ¹⁶
AI/LLM Assistance (Easy Solution Generation)	Focus on Higher-Order Thinking Tasks (Application, Diagnosis) and open-ended projects	Higher-Order Tasks, Unique Assignments ¹⁴
Unauthorized Root Access/Challenge Bypass	Secure Platform Architecture, Logging, and Instructor Review of Submission/Exploit Logs	Submission Log Examination ¹⁶
High-Stakes Pressure Leading to Cheating	Distributing Course Points Across Varied, Frequent, Low-Stakes Assessments	Assessment Design ¹⁵

III. Advancing Web Exploitation Curricula: Modern Stacks and Progressive Difficulty

The existing coursework’s primary reliance on legacy PHP and client-side JavaScript presents a disconnect with contemporary industry practice [Project Plan]. A key objective of this project is to enhance technical relevance by integrating vulnerabilities in widely used, modern languages and frameworks, thereby providing "a greater challenge" with "more modern vulnerabilities" [Project Plan].

3.1. Alignment with Industry Standards and Modern Development

The shift to Node.js, React, and Java is essential for aligning the curriculum with industry standards for scalable web application development.¹⁸ Web security challenges are consistently a core category in professional CTFs because web applications represent the most common attack surface in real-world enterprise environments.⁷ The design of these challenges is systematically mapped to the OWASP Top 10 vulnerabilities, ensuring direct practical application of the knowledge acquired.²

The proposed curriculum structure is designed to introduce vulnerabilities progressively, adhering to the established difficulty taxonomies used in global CTF events, ensuring appropriate instructional scaffolding.

3.2. Technical Deep Dive: Challenge Tiers (Basic to Advanced)

The challenge architecture is categorized into three tiers, moving students from foundational reconnaissance skills to advanced exploit development and chain attacks.

Basic Deliverables (Level 1 Foundation)

These introductory challenges focus on vulnerability identification and basic exploitation techniques, aligning with CTFd Level 1 challenges, such as simple extractions or basic cipher decryption equivalents.¹⁹

- **PHP Legacy Application:** This provides a necessary baseline, focusing on classic SQL Injection (SQLi) with MySQLi to teach students about database interaction flaws and input sanitization [Project Plan].
- **React Single Page App (SPA):** By focusing on a modern React frontend, the learning objective shifts to client-side analysis and the discovery of Document Object Model (DOM) Cross-Site Scripting (XSS). This contrasts starkly with server-side injection, requiring students to use modern browser debugging tools [Project Plan].
- **Node.js/Express:** This introduces students to event-driven architectures. Challenges will focus on insecure middleware, improper routing logic, or file upload flaws, requiring familiarity with Node debugging and request/response injection principles [Project Plan].

Intermediate Deliverables (Level 2 Complex Exploits)

Intermediate challenges require multiple steps or a deeper understanding of framework-specific execution flows, corresponding closely to CTFd Level 2 challenges, which involve command injection or complex debugging analysis.⁷

- **Flask: Server-Side Template Injection (SSTI) and Remote Code Execution (RCE):** SSTI exploitation in Python frameworks like Flask is a highly technical challenge, requiring students to analyze how template engines process untrusted input ²⁰ and subsequently exploit template engine sandbox escapes to achieve RCE.²¹
- **Static File Server: Path Traversal:** A standard Level 2 vulnerability ⁷, this challenge reinforces principles of insecure server configuration and file system navigation, demanding meticulous input manipulation to read arbitrary data.
- **Java Spring Boot (RCE):** The integration of Java Spring Boot is crucial for simulating enterprise environments. To provide a superior educational experience, the challenge should simulate a high-impact, real-world vulnerability such as **Spring4Shell (CVE-2022-22965)**.²² This vulnerability, which affects specific Spring Framework and Spring Boot versions, is critical due to its unauthenticated RCE capability.²³ Exploiting Spring4Shell requires a detailed understanding of the Spring Web MVC architecture, specifically how the `DispatcherServlet` handles requests and the underlying Java class loading mechanisms.²³ By focusing on such a high-profile, critical case study, the educational utility moves beyond mere technical puzzle-solving. It provides a deep, architectural lesson on *why* modern frameworks become vulnerable, directly addressing the project goal of delivering realistic, high-impact challenges.²²

The pedagogical relationship between the project tiers and established difficulty standards is formalized below.

Table 1: Alignment of Project Deliverables with Educational Challenge Taxonomies

Project Deliverable Tier	Key Challenge Examples	CTF Difficulty Level (Based on CTFd Taxonomy)	Core Learning Outcomes (LOs)
Basic	PHP SQLi, React DOM XSS, Node.js Debugging	Level 1 (Simple Extraction, Vigenere equivalent) ¹⁹	Vulnerability Identification, Request/Response Analysis, Client-Side Debugging
Intermediate	Java Spring RCE (Spring4Shell), Flask SSTI/RCE, Path Traversal	Level 2 (Command Injection, GDB analysis equivalent) ¹⁹	Advanced Exploit Crafting, Framework-Specific Vulnerabilities, Template Engine Exploitation
Advanced	Multi-Stage Chain Attacks, Realistic Enterprise Scenario	Level 3+ (Requires tool synthesis and complex logic chaining)	Strategic Thinking, Privilege Escalation, Threat Modelling, Post-Exploitation Techniques ⁶

IV. Constructing Advanced Realistic Scenarios: Multi-Stage Chain Attacks

The Advanced Deliverables mandate a significant step up in complexity, focusing on multi-stage chain attacks within a realistic enterprise scenario [Project Plan]. This design reflects the reality that contemporary cyberattacks rarely rely on a single vulnerability but instead link multiple exploits to achieve deeper compromise.¹²

4.1. Educational Merit of Multi-Stage and Chain Attacks

Designing challenges that require players to complete a full attack chain—such as Reconnaissance, leading to a Foothold, followed by Privilege Escalation, and culminating in objective attainment—is essential for developing strategic and holistic cybersecurity mindsets.¹² This approach compels students to synthesize knowledge from various domains, fostering the complex problem-solving and *pivoting* skills expected of senior security professionals.⁶

The Intermediate tier's introduction of "bread crumbing"—where one successful exploit reveals the necessary information for the next—serves as a vital pedagogical precursor to the full-chain Advanced challenges [Project Plan]. This scaffolding mechanism ensures that students are gradually prepared for the final multi-stage exploit synthesis.⁶ These complex, cumulative final challenges are invaluable, as students who master them are positioned to leverage this knowledge for cutting-edge vulnerability research, including the identification of 0-day flaws in real-world software.⁶

4.2. Instructional Design for Complex Exploits (Modularization)

Complex exploitation, particularly involving concepts like kernel vulnerabilities or heap operations, often creates a massive learning barrier because students must struggle simultaneously with new security concepts and the mastery of intricate diagnostic tools (e.g., `gdb`, `strace`).⁶

To mitigate this, instructional design should prioritize the **modular dissection of complexity**.⁶ Instead of presenting the multi-stage chain as an atomic, monolithic problem, the concepts are split into single-concept modules. This progressive learning structure ensures that students master individual elements sequentially before being required to integrate them concurrently for the final chain attack. This measured progression minimizes conceptual jumps and maintains the requisite educational approach over a frustrating, challenge-focused CTF design.⁶

A key instructional technique is the decoupling of concept mastery from tool mastery. This is achieved through the use of **self-guiding challenge variants** in the preparatory modules.⁶ For example, in a module focusing on buffer overflows, the self-guiding variant might automatically implement breakpoints or display disassembled payload code, allowing the student to focus entirely on understanding the core concept. Only once the concept is mastered does the student transition to the regular variant, where they must apply their own learned analysis tools (like `gdb`) to replicate the solution.⁶

Finally, achieving realism in the Advanced tier necessitates simulating a practical enterprise environment. This involves incorporating elements such as layered defense, specific network segmentation, and post-exploitation techniques (e.g., lateral movement, infrastructure compromise).¹² Simulating these complex, real-world constraints ensures the advanced challenges accurately reflect the demands of a professional Red Team engagement.

V. Evaluation and Measurement of Learning Gain in CTF Coursework

The project includes an opportunity to collect data from willing student participants in the January–February 2026 module [Project Plan]. Leveraging this opportunity requires defining rigorous, quantifiable metrics to accurately measure the educational efficacy and the resulting student learning gain.

5.1. Quantifying Educational Effectiveness (Learning Gain)

Learning gain is defined as the quantifiable "distance travelled" in a student's performance between the initiation and completion of their studies.²⁶ To rigorously validate the CTF module redesign as an effective educational intervention, the project must employ the **normalized gain** metric.²⁷

Calculating normalized gain requires administering objective pre- and post-module assessments (encompassing both cognitive knowledge and technical skills).²⁷ The normalized gain metric accounts for students' prior knowledge, providing a statistically sound measure of how effective the intervention was in transferring knowledge. Previous academic interventions utilizing similar hands-on methodologies have demonstrated statistically significant improvements, with technical skills showing high average normalized gains, corroborating the effectiveness of CTF-based teaching.²⁷

5.2. Metrics for Assessing Challenge Efficacy and Performance Data

Objective performance data collected directly from the CTF platform (e.g., CTFd logs) provides granular measures of student proficiency and challenge efficacy. This approach utilizes modified security metrics, adapting industry Key Performance Indicators (KPIs) like Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR).²⁸

- 1. **Time-to-Flag (T2F):** This metric measures the elapsed time from when a student initiates a challenge until they successfully submit the correct flag. T2F serves as an objective measure of efficiency and skill proficiency.²⁹
- 2. **Attempt and Failure Rate:** Monitoring the number of failed flag submissions per challenge provides insight into student persistence and the inherent difficulty balance of the challenge.¹³ Analyzing these attempt counts and mapping them to Csikszentmihalyi's flow model can identify challenges that are poorly calibrated—either too frustrating or insufficiently challenging—allowing for future refinement.¹³
- 3. **Hint Usage:** If instructional hints are provided, tracking their frequency and strategic deployment indicates whether students require conceptual direction or minor navigational assistance, informing both future challenge design and supporting resource allocation.²⁵

5.3. Qualitative Feedback and Data Collection Logistics

Quantitative metrics must be supplemented by robust qualitative data to assess student perceptions, confidence shifts, and motivational factors, ideally structured around established models such as the ARCS model of motivational design.³

Methods for collecting this subjective data include:

- **Anonymized Electronic Questionnaires:** These formal surveys monitor and enhance the quality of learning by gathering student opinions on module content and delivery.³⁰
- **Informal Feedback:** Channels such as open forums or direct contact with instructors provide timely feedback on issues requiring immediate resolution.³⁰

To ensure academic rigor, particularly in high-stakes assessment environments where AI assistance is a risk, measuring only the binary success of finding the flag is insufficient. The project must implement a system for capturing *how* the student arrived at the solution.²⁵ For advanced or intermediate challenges, this necessitates collecting a mandatory post-submission component, such as a brief write-up or detailed explanation of the exploit methodology.³¹ This assesses the student's critical thinking, originality, and demonstration of a structured problem-solving methodology, which serves as a crucial check against simple copy-pasting of solutions.

The empirical success of the dissertation project hinges upon the systematic collection and analysis of these metrics, ensuring the evaluation process adheres to university ethical research guidelines.

Table 3: Recommended Performance Metrics for CTF Learning Gain Assessment (Jan-Feb 2026 Data Collection)

Metric Category	Recommended Metric (Adapted for CTF)	Pedagogical Insight Measured	Literature Support
Knowledge Transfer	Normalized Gain (Pre- vs. Post-Test Scores)	Effectiveness of Educational Intervention and knowledge acquisition	Absolute/Relative Gain ²⁷
Efficacy & Skill	Time-to-Flag (T2F) Completion Time	Efficiency and Skill Proficiency (Adapted from MTTD/MTTR)	Cybersecurity KPIs ²⁸
Difficulty Balance	Failure Rate per Challenge/Attempt Count	Challenge Efficacy and Flow State Analysis (Avoiding boredom/anxiety)	Flow Model ¹³
Conceptual Depth	Quality of Post-Submission Write-ups/Explanations	Critical Thinking, Methodology Demonstration, and Originality	Assessment/Portfolio Development ²⁵

VI. Conclusion: Synthesis and Framework Development

The literature review provides a robust justification for the proposed redesign of the Networks and Systems cybersecurity module, establishing a framework based on modern pedagogy, advanced integrity measures, and technical relevance.

6.1. Synthesis of Core Findings

The analysis confirms that the adoption of CTF-style challenges is pedagogically sound, driving engagement and leading to superior skill refinement by forcing the practical application of underlying Computer Science theory. However, the existing coursework's uniform challenge structure poses critical risks to academic integrity from peer-to-peer sharing and AI augmentation. Addressing this necessitates a fundamental architectural change, moving toward dynamically generated, personalized lab environments (SecGen methodology) and assessment designs that prioritize higher-order thinking over simple solution recognition.

Technically, the transition from legacy PHP to modern stacks—Node.js, React, and Java Spring Boot—is critical for preparing students for the enterprise security landscape. The tiered difficulty structure, ranging from foundational SQLi to complex, framework-specific RCE vulnerabilities like Spring4Shell, ensures structured progression. The Advanced tier is justified by the necessity of training strategic thinking through multi-stage chain attacks, reflecting realistic threat actor behavior and serving as a foundation for future vulnerability research.

6.2. Recommendations and Future Work

The successful implementation of this dissertation project will establish a modular, challenging, and defensible CTF coursework framework. The core challenge of the original module—the identity of the challenges—is mitigated in this project by varying credentials, ports, and configuration strength within the custom web challenges. However, the comprehensive randomization of the *entire* 23-problem VM structure remains an extensive architectural undertaking [Project Plan].

Therefore, while this project significantly increases the difficulty and relevance of the web exploitation sub-module, future work should focus on developing a full, parameterized CTF generation engine capable of dynamically generating all 23 challenge problems based on a unique student seed. Such an effort would require substantial infrastructure commitment to maintain the complexity, scalability, and randomization across all categories (Pwning, Crypto, Forensics) for long-term pedagogical sustainability. The empirical data collected from the optional student participation in 2026, analyzed through the Normalized Gain metric $\angle g \angle$, will provide the critical evidence needed to further justify such expansive architectural improvements.

References

- 1 A. Leune and M. Petrilli, "Using Capture-the-Flag to Enhance the Effectiveness of Cybersecurity Education," ResearchGate, 2017.
- 2 A. Al-Rawi and M. Al-Thahab, "Web Application Security Course based on OWASP Top 10 Using the Hands-on Approach," MDPI Electronics, vol. 11, no. 21, p. 3517, 2022.
- 3 S. Karagiannis and E. Magos, "Adapting CTF challenges into virtual cybersecurity learning environments," ResearchGate, 2020.
- 4 J. O. Williams, A. Wong, and Y. Khmelevsky, "Engaging underrepresented students in cybersecurity using Capture-the-Flag (CTF) competitions: Experience," ASEE Peer, 2020.
- 5 A. Williams, E. P. Ferrari, and A. Wong, "Leaderboard effects, student engagement and learning outcomes in a cybersecurity intervention," ACM SIGCSE '21, 2021.
- 6 C. Yan and F. Li, "PWN the Learning Curve: A Pedagogical Approach to Teaching Advanced Software Exploitation," SIGCSE, 2024.
- 7 CTFd Documentation. (n.d.). "CTF Challenge Categories."
- 8 Parrot CTFs. (n.d.). "The Complete Guide to CTF Event Hosting, Planning, Platforms, and Best Practices."
- 9 Institute of Data. (n.d.). "Capture the Flag in Cybersecurity."
- 10 Salesforce Engineering. (n.d.). "Capture the Flag: Secure Your Knowledge."
- 11 Parrot CTFs. (n.d.). "The Complete Guide to Cybersecurity Learning Platforms in 2025."
- 12 Parrot CTFs. (2023). "CTF Training Programs for Universities."
- 13 NCSC. (2022). "Effectiveness of CTF education." The Hague University of Applied Sciences.
- 14 UCF Center for Distributed Learning. (n.d.). "Promoting Academic Integrity with AI Tools like ChatGPT."
- 15 Ohio State University. (n.d.). "Design Assessments That Reduce Motivation and Opportunity for Cheating."

- 16 T. Chothia and S. Novakovic, "Preventing Cheating in Hands-on Lab Assignments," ResearchGate, 2023.
- 17 Ohio State University. (n.d.). "Strategies & Tools for Academic Integrity."
- 18 Board Infinity. (n.d.). "Libraries and Frameworks for Frontend Development (Coursera)."
- 19 CTFd Documentation. (n.d.). "CTF Challenge Levels."
- 20 YesWeHack. (n.d.). "Server-Side Template Injection Exploitation."
- 21 GitHub Repository. (n.d.). "Server-Side Template Injection Payloads."
- 22 Snyk Learn. (n.d.). "Spring4Shell (CVE-2022-22965)."
- 23 Hack The Box. (2022). "Spring4Shell Explained: CVE-2022-22965."
- 24 W. Ma, et al., "Anomaly based multi-stage attack detection method," PLOS ONE, vol. 19, no. 3, 2024.
- 25 C. Yan and F. Li, "PWN the Learning Curve: A Pedagogical Approach to Teaching Advanced Software Exploitation," SIGCSE, 2024 (Discussion on multi-stage exploits).
- 26 RAND Europe. (n.d.). "Learning gain in higher education."
- 27 D. R. Ragsdale, "Learning gain and retention in a hands-on laboratory approach to teaching introductory engineering," PMC Articles, 2011.
- 28 SentinelOne. (n.d.). "Cybersecurity Metrics & KPIs: What to Track in 2025."
- 29 Teradata. (n.d.). "Security Metric FAQs."
- 30 The Office of the Registrar. (n.d.). "CTF Student Evaluation Policy," University of Cambridge.
- 31 Parrot CTFs. (n.d.). "From CTF Player to Security Professional."